

PEC 2: Juegos

Presentación

Segunda PEC del curso de Inteligencia Artificial

Competencias

En esta PEC se trabajarán las siguientes competencias:

Competencias de grado:

- Capacidad de analizar un problema con el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y solucionarlo.

Competencias específicas:

- Saber representar las particularidades de un problema según un modelo de representación del conocimiento.
- Saber resolver problemas intratables a partir de razonamientos aproximados y heurísticos (algoritmos voraces, algoritmos genéticos, lógica difusa, redes bayesianas, redes neuronales, min-max).

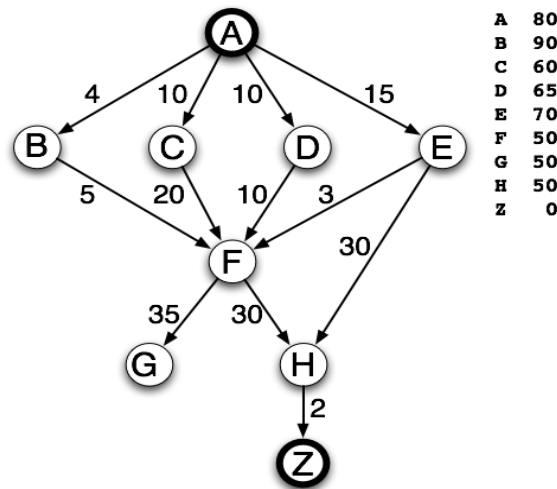
Objetivos

Esta PEC pretende evaluar vuestros conocimientos sobre búsquedas, su formalización y estrategias relacionadas.

Descripción de la PEC/práctica a realizar

Parte 1 (40%)

¿Qué camino encontrará el algoritmo A* en este grafo dirigido? El nodo inicial es A y el nodo final objetivo es Z. A cada arista se le ha asignado un coste y en la tabla adjunta se encontrará la función heurística (el coste estimado para llegar del nodo a Z).



Detalle el proceso paso a paso especificando qué nodo se expande en cada momento y las consecuencias que se derivan para los caminos encontrados hasta el momento.

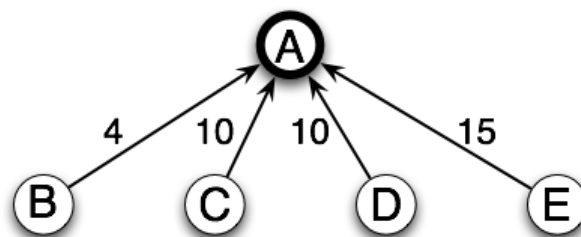
Solución:

Tendremos dos listas: abiertos y cerrados, para ir guardando los nodos procesados. Para cada nodo se detallará el valor de su función $f = g+h$, con el mejor camino encontrado hasta el momento. A cada paso diremos qué nodo se expande, por tener el coste mínimo en la lista de abiertos, y haremos hincapié en las posibles redirecciones provocadas en cada nodo debido a la aparición de nuevos nodos.

Inicialmente:

abiertos = {A (0+80)} cerrados = {}

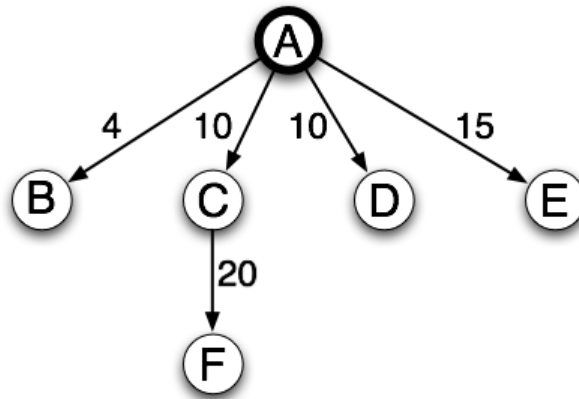
1.- Exp. A (0+80) → Suc. B (4+90), C (10+60), D (10+65), E (15+70)



abiertos = {B (4+90), C (10+60), D (10+65), E (15+70)}

cerrados = {A (0+80)}

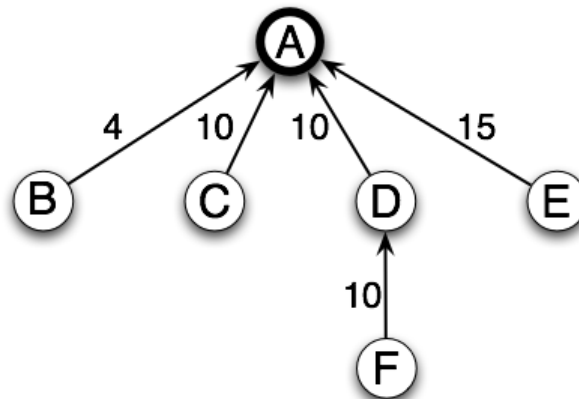
2.- Exp. C (10+60) → Suc. F (10+20+50)



abiertos = {B (4+90), D (10+65), E (15+70), F (10+20+50)}

cerrados = {A (0+80), C (10+60)}

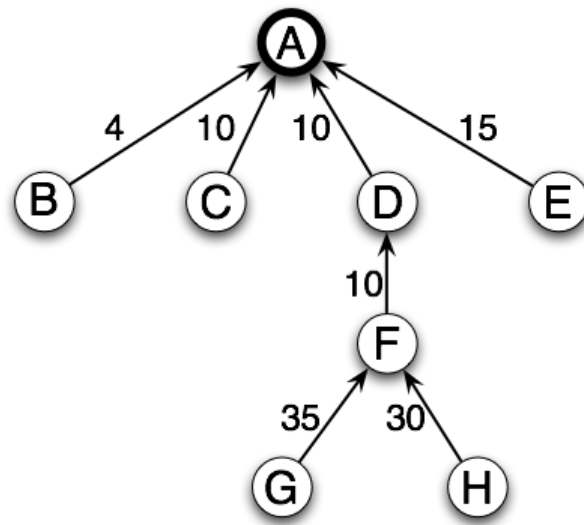
3.- Exp. D (10+65) → Suc. F (10+10+50)



abiertos = {B (4+90), E (15+70), F (10+10+50)}

cerrados = {A (0+80), C (10+60), D (10+65)}

4.- Exp. F (10+10+50) → Suc. G (10+10+35+50), H (10+10+30+50)



abiertos = {B (4+90), E (15+70), G (10+10+35+50), H (10+10+30+50)}

cerrados = {A (0+80), C (10+60), D (10+65), F (10+10+50)}

5.- Exp. E (15+70) → Suc. F (15+3+50), H (15+30+50)

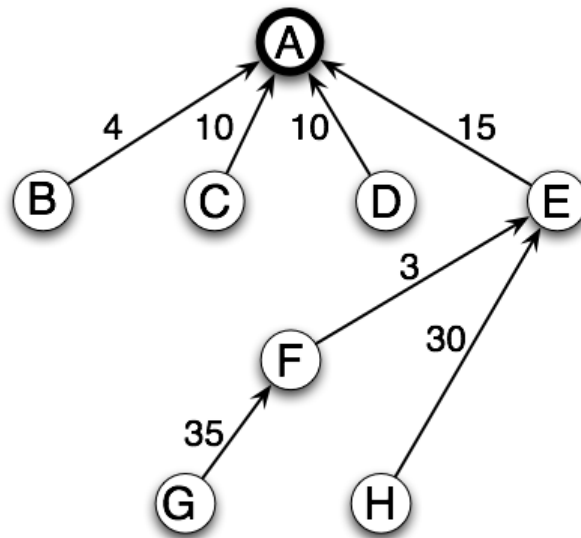
Hemos encontrado una F y una H mejores que las que teníamos, por tanto, como F está en cerrados, también hay que revisar sus sucesores, los sucesores de los sucesores, etc.

F (10+10+50) a F (15+3+50)

Sucesores de F: G y H

G (10+10+35+50) a G (15+3+35+50)

H (10+10+30+50) a H (15+3+30+50) pero esta NO es mejor que la que ya tenemos



abiertos = {B (4+90), G (15+3+35+50), H (15+30+50)}

cerrados = {A (0+80), C (10+60), D (10+65), F (15+3+50), E (15+70)}

6.- Exp. B (4+90) → Suc. F (4+5+50)

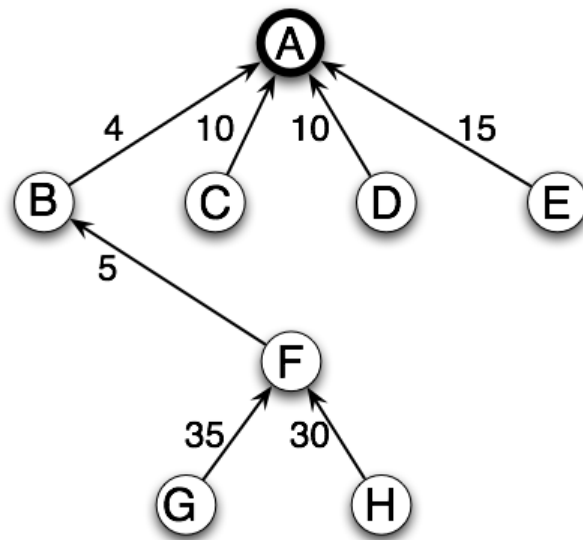
Hemos encontrado una F mejor que la que teníamos, por tanto, como F está en cerrados, también hay que revisar sus sucesores, los sucesores de los sucesores, etc.

F (15+3+50) a F (4+5+50)

Sucesores de F: G y H

G (15+3+35+50) a G (4+5+35+50)

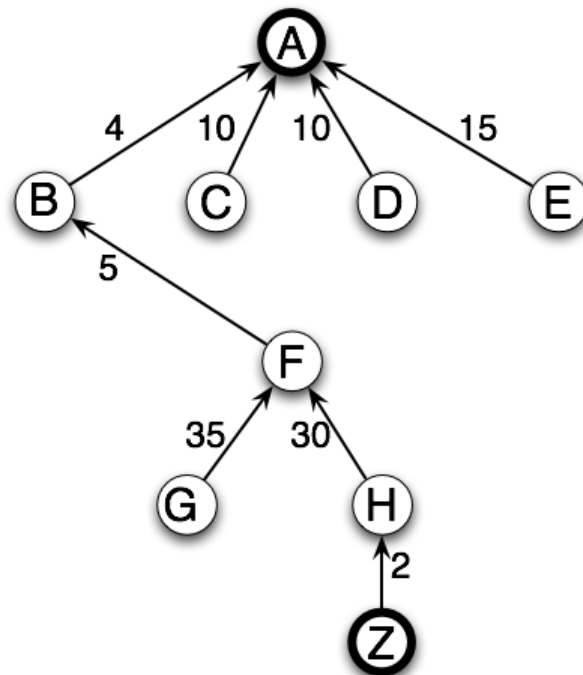
H (15+3+30+50) a H (4+5+30+50)



abiertos = {G (4+5+35+50), H (4+5+30+50)}

cerrados = {A (0+80), C (10+60), D (10+65), F (4+5+50), E (15+70), B (4+90)}

7.- Exp. H (4+5+30+50) → Suc. Z (4+5+30+2)



abiertos = {G (4+5+35+50), Z (4+5+30+2)}

cerrados = {A (0+80), C (10+60), D (10+65), F (4+5+50), E (15+70), B (4+90),
H (4+5+30+50)}

8.- Exp. Z (4+5+30+2) y acabamos. Camino encontrado: A-B-F-H-Z con coste 41.

Parte 2 (4 x 15%)

En el Módulo 2 hay una implementación en Common Lisp de un sistema de búsqueda general que con pocas adaptaciones se puede transformar en diferentes tipos de búsquedas. Así, el código del sistema general está descrito en los apartados 1.1.3 y 2.1 y tiene la búsqueda en anchura definida en el apartado 3.1.1, la búsqueda en profundidad en el apartado 3.2, la búsqueda en profundidad limitada en el apartado 3.2.1 y la búsqueda A* a partir del apartado 4.3.2. Para ilustrar el código, éste está aplicado al ejemplo del rompecabezas lineal.

En esta PEC2 tenéis este código todo junto en un fichero `astar.lisp` que adjuntamos con la PEC2. Allí está todo el sistema del Módulo 2 implementado (con un par de modificaciones indicadas en el código, para hacer más comprensible el resultado) con énfasis en la búsqueda A*. En el archivo mencionado está parte de la descripción necesaria para aplicarlo al grafo de la parte 1 de esta PEC2. El archivo, sin embargo, está incompleto. A esta parte de la PEC2 vosotros añadiréis lo necesario para completarlo.

a) (15%) Define una función heurística que reciba como parámetro un estado y devuelva el coste estimado de ir de aquel estado al estado final, correspondiente a la función heurística de la parte 1. Un estado es un símbolo: **A, B, C**, etc.

(defun heuristica (estado) . . .)

```
(defun heuristica (estado)
  (cond ((equal estado 'A)      80)
        ((equal estado 'B)      90)
        ((equal estado 'C)      60)
        ((equal estado 'D)      65)
        ((equal estado 'E)      70)
        ((equal estado 'F)      50)
        ((equal estado 'G)      50)
        ((equal estado 'H)      50)
        ((equal estado 'Z)       0)
        (t                       100)))
```

b) (15%) Una vez ya tenemos definida la función heurística, también hay que definir los costes asociados a ir de un estado a otro. Así pues, define la función `coste` que devuelva el coste entre dos estados que estén conectados en el grafo de la parte 1:

(defun coste (estado1 estado2) . . .)

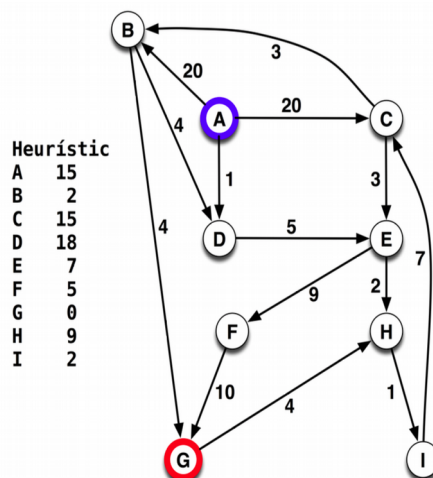
```
(defun coste (estado1 estado2)
  (cond ((and (equal estado1 'A) (equal estado2 'B)) 4)
        ((and (equal estado1 'A) (equal estado2 'C)) 10)
        ((and (equal estado1 'A) (equal estado2 'D)) 10)
```

```
((and (equal estado1 'A) (equal estado2 'E)) 15)
((and (equal estado1 'B) (equal estado2 'F)) 5)
((and (equal estado1 'C) (equal estado2 'F)) 20)
((and (equal estado1 'D) (equal estado2 'F)) 10)
((and (equal estado1 'E) (equal estado2 'F)) 3)
((and (equal estado1 'E) (equal estado2 'H)) 30)
((and (equal estado1 'F) (equal estado2 'G)) 35)
((and (equal estado1 'F) (equal estado2 'H)) 30)
((and (equal estado1 'H) (equal estado2 'Z)) 2)
(t 100)))
```

c) (15%) Comprueba que la resolución de la parte 1 es correcta utilizando el código dado conjuntamente con las 2 funciones que has programado en los apartados 2-a y 2-b. Para ello será necesario que utilices la función `busqueda-A*` que ya viene dada. Especifica qué llamada haces y qué resultado devuelve el programa.

(busqueda-a* problema-busquedaA*) --> (AtoB BtoF FtoH HtoZ)

d) (15%) Utilizando el código del archivo adjunto, resuelve ahora este nuevo problema de búsqueda A* (A es el estado de partida, G estado objetivo) ¿La solución es óptima? ¿Qué modificaciones del código han sido necesarias?



Hay que definir los operadores, la variable `tl-operadores`, y las funciones `(coste e1 e2)` y `(heurística e)`. Así definimos el grafo. Además, hay que definir en `problema-busquedaA*` cuáles serán el estado inicial y el estado final. El resto del código es propio del algoritmo A*, por tanto no hay que cambiarlo. Si hacemos

(busqueda-a* problema-busquedaA*) --> (AtoB BtoG)

Como el heurístico es no admisible la solución puede no ser óptima. Y eso es lo que pasa. Véase el archivo adjunto `astar2.lisp`.

Recursos

Para realizar esta PEC el material imprescindible los temas 2, 3 y 4 del Módulo 2.

Criterios de valoración

Indicados en el enunciado.

Formato y fecha de envío

Para dudas y aclaraciones sobre el enunciado, dirigiros al consultor responsable del aula.

Hay que entregar la solución en un archivo **PDF**. Adjuntar el fichero a un mensaje en el apartado Entrega y Registro de EC (REC).

El nombre del archivo debe ser Apellidos_Nombre_IA_PEC2 con la extensión. pdf (PDF).

La fecha límite de entrega es el: **10 de Noviembre** (a las 24 horas, más o menos).

Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.

Nota: **Propiedad intelectual**

A menudo es inevitable, al producir una obra multimedia, hacer uso de recursos creados por terceras personas. Es por tanto comprensible hacerlo en el marco de una práctica de los estudios de Informática, siempre que se documente claramente y no suponga plagio en la práctica.

Por lo tanto, al presentar una práctica que haga uso de recursos ajenos, se presentará junto con ella un documento en el que se detallen todos ellos, especificando el nombre de cada recurso, su autor, el lugar donde se obtuvo y el su estatus legal: si la obra está protegida por copyright o se acoge a alguna otra licencia de uso (Creative Commons, licencia GNU, GPL ...). El estudiante deberá asegurarse de que la licencia que sea no impide específicamente su uso en el marco de la práctica. En caso de no encontrar la información correspondiente deberá asumir que la obra está protegida por copyright. Deberán, además, adjuntar los archivos originales cuando las obras utilizadas sean digitales, y su código fuente esté corresponde.